

Step 1: Start Mini Kube

```
MINGW64:/c/Users/jagad/gitrepos/te-5th_and_6th_batch_assignments/surinenijagadeesh/Kubernetes-Assignment-1

jagad@SURINENI MINGW64 ~
$ minikube start
* minikube v1.38.1 on Microsoft Windows 11 Home Single Language 25H2
* Using the docker driver based on existing profile
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.50 ...
* Restarting existing docker container for "minikube" ...
* Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
* Verifying Kubernetes components...
- Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Enabled addons: storage-provisioner, default-storageclass
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
```

This command starts the Mini Kube Kubernetes cluster. It prepares the local Kubernetes environment so we can create and manage Pods.

Step 2: Check Mini Kube Status

```
jagad@SURINENI MINGW64 ~
$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
```

This command checks whether the Mini Kube cluster is running. It shows the status of the host, kubelet, API server, and kubeconfig.

Step 3: Check Kubernetes Node

```
jagad@SURINENI MINGW64 ~
$ kubectl get nodes
NAME        STATUS    ROLES    AGE   VERSION
minikube    Ready     control-plane  2d9h  v1.35.1
```

This command displays the nodes in the Kubernetes cluster. It confirms that the Mini Kube node is Ready and the cluster is working properly.

Step 4: Create the Pod Manifest (YAML) File

```
MINGW64:/c/Users/jagad/gitrepos/te-5th_and_6th_batch_assignments/surinenijagadeesh/Kubernetes-Assignment-1
jagad@SURINENI MINGW64 ~/gitrepos/te-5th_and_6th_batch_assignments/surinenijagadeesh/Kubernetes-Assignment-1 (main)
$ vim jagadeesh-nginx-pod.yaml

jagad@SURINENI MINGW64 ~/gitrepos/te-5th_and_6th_batch_assignments/surinenijagadeesh/Kubernetes-Assignment-1 (main)
$ ls
jagadeesh-nginx-pod.yaml

jagad@SURINENI MINGW64 ~/gitrepos/te-5th_and_6th_batch_assignments/surinenijagadeesh/Kubernetes-Assignment-1 (main)
$ cat jagadeesh-nginx-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: jagadeesh-nginx-pod
spec:
  containers:
  - name: jagadeesh-container
    image: nginx
    ports:
    - containerPort: 80
```

This command creates a YAML file for the Kubernetes Pod. The YAML file contains the Pod configuration, such as the Pod name, container name, image, and port.

Step 5: Create and Verify the Pod

```
jagad@SURINENI MINGW64 ~/gitrepos/te-5th_and_6th_batch_assignments/surinenijagadeesh/Kubernetes-Assignment-1 (main)
$ kubectl apply -f jagadeesh-nginx-pod.yaml
pod/jagadeesh-nginx-pod created

jagad@SURINENI MINGW64 ~/gitrepos/te-5th_and_6th_batch_assignments/surinenijagadeesh/Kubernetes-Assignment-1 (main)
$ kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
jagadeesh-nginx-pod 0/1     ContainerCreating 0          11s

jagad@SURINENI MINGW64 ~/gitrepos/te-5th_and_6th_batch_assignments/surinenijagadeesh/Kubernetes-Assignment-1 (main)
$ kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
jagadeesh-nginx-pod 1/1     Running   0          2m58s
```

kubectl apply -f jagadeesh-nginx-pod.yaml : This command creates the Pod using the YAML file.

kubectl get pods: This command checks the Pod status. First, the Pod was in Container Creating state, and after a few seconds, it changed to Running. This means the Pod was created successfully.

Step 6: Describe the Pod

```
MINGW64/c/Users/jagad/gitrepos/te-5th_and_6th_batch_assignments/surinenijagadeesh/Kubernetes-Assignment-1
jagad@SURINENI MINGW64 ~/gitrepos/te-5th_and_6th_batch_assignments/surinenijagadeesh/Kubernetes-Assignment-1 (main)
$ kubectl describe pod jagadeesh-nginx-pod
Name: jagadeesh-nginx-pod
Namespace: default
Priority: 0
Service Account: default
Node: minikube/192.168.49.2
Start Time: Sun, 02 Aug 2026 17:58:38 +0530
Labels: <none>
Annotations: <none>
Status: Running
IP: 10.244.0.4
IPs:
  IP: 10.244.0.4
Containers:
  jagadeesh-container:
    Container ID: docker://892b24126763df6452a7a8a9bb251e6d9bb1a76ebd44a1066af8e747f265bf35
    Image: nginx
    Image ID: docker-pullable://nginx@sha256:5a88c9c45479443d7be2eadc894b4ed0a9801bae03d97a5760ae13b5c2005942
    Port: 80/TCP
    Host Port: 0/TCP
    State: Running
      Started: Sun, 02 Aug 2026 17:59:05 +0530
    Ready: True
    Restart Count: 0
    Environment: <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-nnfqf (ro)
Conditions:
  Type              Status
  PodReadyToStartContainers  True
  Initialized         True
  Ready              True
  ContainersReady     True
  PodScheduled       True
Volumes:
  kube-api-access-nnfqf:
    Type: Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds: 3607
    ConfigMapName: kube-root-ca.crt
    Optional: false
    DownwardAPI: true
QoS Class: BestEffort
Node-Selectors: <none>
Tolerations: node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
              node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type    Reason      Age   From          Message
  ----    -
  Normal  Scheduled   3m55s  default-scheduler  Successfully assigned default/jagadeesh-nginx-pod to minikube
  Normal  Pulling     3m55s  kubelet        spec.containers{jagadeesh-container}: Pulling image "nginx"
  Normal  Pulled      3m30s  kubelet        spec.containers{jagadeesh-container}: Successfully pulled image "nginx" in 25.078s (25.078s including waiting). Image size: 161327548 bytes.
  Normal  Created     3m29s  kubelet        spec.containers{jagadeesh-container}: Container created
  Normal  Started     3m29s  kubelet        spec.containers{jagadeesh-container}: Container started
```

This command displays detailed information about the Pod, such as its status, IP address, container details, and recent events. It is useful for checking the Pod and troubleshooting issues.

Step 7: View Pod Details

```
jagad@SURINENI MINGW64 ~/gitrepos/te-5th_and_6th_batch_assignments/surinenijagadeesh/Kubernetes-Assignment-1 (main)
$ kubectl get pods -o wide
NAME                                STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES
jagadeesh-nginx-pod                1/1      Running    0     10.244.0.4    minikube   <none>           <none>

jagad@SURINENI MINGW64 ~/gitrepos/te-5th_and_6th_batch_assignments/surinenijagadeesh/Kubernetes-Assignment-1 (main)
$ kubectl logs jagadeesh-nginx-pod
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/08/02 12:29:05 [notice] 1#1: using the "epoll" event method
2026/08/02 12:29:05 [notice] 1#1: nginx/1.31.3
2026/08/02 12:29:05 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/08/02 12:29:05 [notice] 1#1: OS: Linux 6.18.33.2-microsoft-standard-WSL2
2026/08/02 12:29:05 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/08/02 12:29:05 [notice] 1#1: start worker processes
2026/08/02 12:29:05 [notice] 1#1: start worker process 29
2026/08/02 12:29:05 [notice] 1#1: start worker process 30
2026/08/02 12:29:05 [notice] 1#1: start worker process 31
2026/08/02 12:29:05 [notice] 1#1: start worker process 32
2026/08/02 12:29:05 [notice] 1#1: start worker process 33
2026/08/02 12:29:05 [notice] 1#1: start worker process 34
2026/08/02 12:29:05 [notice] 1#1: start worker process 35
2026/08/02 12:29:05 [notice] 1#1: start worker process 36
```

kubectl get pods -o wide : This command shows the Pod with additional details like the Pod IP address and the node where it is running.

kubectl logs jagadeesh-nginx-pod : This command displays the logs of the running container. It helps verify that the Nginx application started successfully and is running without errors.

Step 8: Access the Pod (kubectl exec)

```
2020/08/02 12:29:05 [notice] 1/1: Start worker process 30
jagad@SURINENI MINGW64 ~/gitrepos/te-5th_and_6th_batch_assignments/surinenijagadeesh/Kubernetes-Assignment-1 (main)
$ kubectl exec -it jagadeesh-nginx-pod -- sh
# pwd
/
# ls
bin boot dev docker-entrypoint.d docker-entrypoint.sh etc home lib lib64 media mnt opt proc root run sbin srv sys tmp usr var
#
```

This command is used to access the running container inside the Pod. After entering the container, I used `pwd` and `ls` to check the current directory and available files.

Step 9: Delete the Pod

```
jagad@SURINENI MINGW64 ~/gitrepos/te-5th_and_6th_batch_assignments/surinenijagadeesh/Kubernetes-Assignment-1 (main)
$ kubectl delete pod jagadeesh-nginx-pod
pod "jagadeesh-nginx-pod" deleted from default namespace

jagad@SURINENI MINGW64 ~/gitrepos/te-5th_and_6th_batch_assignments/surinenijagadeesh/Kubernetes-Assignment-1 (main)
$ kubectl get pods
No resources found in default namespace.

jagad@SURINENI MINGW64 ~/gitrepos/te-5th_and_6th_batch_assignments/surinenijagadeesh/Kubernetes-Assignment-1 (main)
$
```

This command deletes the Pod from the Kubernetes cluster. After deleting the Pod, I verified it using `kubectl get pods`, and no Pods were found in the default namespace.

Pod Lifecycle

Explanation: The Pod lifecycle describes the different stages a Pod goes through from creation to termination.

1. Pending: The Pod is created and waiting to be scheduled or download the container image.
2. Running: The Pod is running successfully, and the container is active.
3. Succeeded: The Pod has completed its work successfully.
4. Failed: The Pod stopped because of an error.

CrashLoopBackOff

CrashLoopBackOff means the container keeps crashing, and Kubernetes keeps restarting it.

It occurs when the application inside the container fails to start or crashes repeatedly.

Common Reasons

Wrong container image

Application error

Incorrect configuration

Memory or resource issues

Troubleshooting

Check the Pod logs using `kubectl logs`.

Check Pod details using `kubectl describe pod`.

Verify the image and configuration.

Fix the application error and recreate the Pod.